Tasks on K8S-04

1. Deploy an application using a Deployment with 3 replicas and a rolling update strategy.

Objective: Create a Deployment with three Nginx replicas using the RollingUpdate strategy.

Implementation Steps:

1. Connect to the Kubernetes master node.
2. Create a YAML file:

   vi first-deployment.yaml
3. Add the following configuration:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: first-deployment
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.26
        ports:
```

- containerPort: 80

4. Save the file and create the Deployment:

   kubectl apply -f first-deployment.yaml

5. Verify the Deployment:

   kubectl get deployments

6. Verify the Pods:

   kubectl get pods

7. Check the Deployment details:

   kubectl describe deployment first-deployment

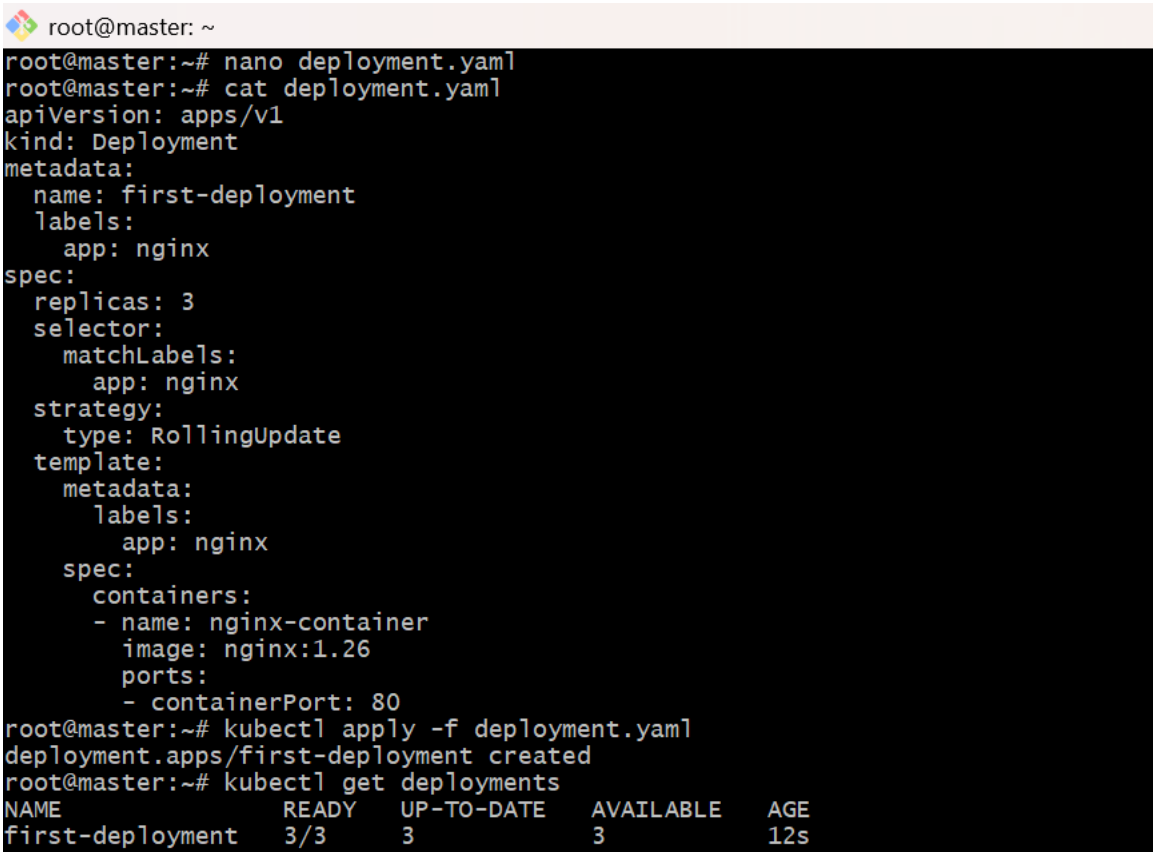
8. Check the ReplicaSets:

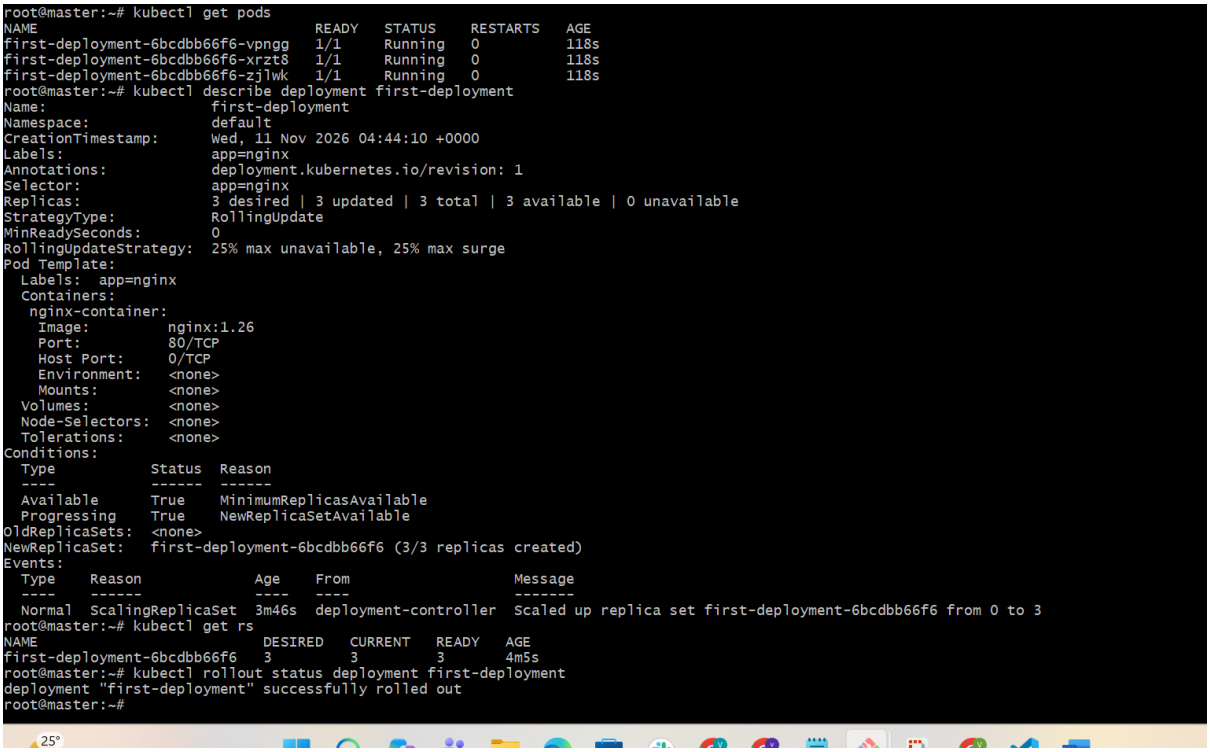
   kubectl get replicasets

9. Check the rollout status:

   kubectl rollout status deployment/first-deployment

Expected Result: Three Nginx Pods should be created successfully. During an image update, Kubernetes replaces old Pods gradually using the RollingUpdate strategy.





2. Configure a Deployment with a Recreate strategy and observe the downtime.

Objective: Create a Deployment using the Recreate strategy and observe how Kubernetes replaces Pods during an update.

Implementation Steps:

1. Create a YAML file:

   vi recreate-deployment.yaml

2. Add the following configuration:

apiVersion: apps/v1

kind: Deployment

metadata:

 name: recreate-deployment

spec:

 replicas: 3

 strategy:

  type: Recreate

 selector:

  matchLabels:

```
    app: recreate-nginx
  template:
    metadata:
      labels:
        app: recreate-nginx
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.26
        ports:
          - containerPort: 80
```

3. Save the file and create the Deployment:

   kubectl apply -f recreate-deployment.yaml

4. Verify the Deployment and Pods:

5. kubectl get deployments

   kubectl get pods

6. Open another terminal and monitor the Pods:

   kubectl get pods -w

7. Update the container image from Nginx to Apache:

   kubectl set image deployment/recreate-deployment nginx-container=httpd:2.4

8. Observe the Pod replacement process in the monitoring terminal.

9. Verify the updated image:

   kubectl describe deployment recreate-deployment

10. Check the rollout status:

    kubectl rollout status deployment/recreate-deployment

11. Verify the newly created Pods:

    kubectl get deployment first-deployment

Expected Result: Kubernetes terminates all existing Pods before creating new Pods with the updated image. During this process, the application may experience downtime because the old Pods are removed before the new Pods become available.

```
root@master:~# nano recreate-deployment.yaml
root@master:~# cat recreate-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: recreate-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: recreate-nginx
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: recreate-nginx
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.26
        ports:
        - containerPort: 80
root@master:~# kubectl apply -f recreate-deployment.yaml
deployment.apps/recreate-deployment created
root@master:~# kubectl get deployments
kubectl get pods
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
first-deployment      3/3     3            3           7m26s
recreate-deployment   3/3     3            3           13s
NAME                                   READY   STATUS    RESTARTS   AGE
first-deployment-6bcdbb66f6-vpngg      1/1     Running   0          7m26s
first-deployment-6bcdbb66f6-xrzt8      1/1     Running   0          7m26s
first-deployment-6bcdbb66f6-zjlwk      1/1     Running   0          7m26s
recreate-deployment-5447796b44-8rwv9   1/1     Running   0          13s
recreate-deployment-5447796b44-bdbdj   1/1     Running   0          13s
recreate-deployment-5447796b44-mkwgc   1/1     Running   0          13s
```

```
root@master:~# nano recreate-deployment.yaml
root@master:~# cat recreate-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: recreate-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: recreate-nginx
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: recreate-nginx
    spec:
      containers:
      - name: nginx-container
        image: httpd:2.4
        ports:
        - containerPort: 80
root@master:~# kubectl apply -f recreate-deployment.yaml
deployment.apps/recreate-deployment configured
root@master:~#
```

```
root@master:~#
root@master:~# kubectl get pods -w
NAME                                    READY   STATUS    RESTARTS   AGE
first-deployment-6bcdbb66f6-vpngg       1/1     Running   0          9m19s
first-deployment-6bcdbb66f6-xrzt8       1/1     Running   0          9m19s
first-deployment-6bcdbb66f6-zjlwk       1/1     Running   0          9m19s
recreate-deployment-5447796b44-8rwv9    1/1     Running   0          2m6s
recreate-deployment-5447796b44-bdbdj    1/1     Running   0          2m6s
recreate-deployment-5447796b44-mkwgc    1/1     Running   0          2m6s
recreate-deployment-5447796b44-mkwgc    1/1     Terminating   0          3m43s
recreate-deployment-5447796b44-8rwv9    1/1     Terminating   0          3m43s
recreate-deployment-5447796b44-bdbdj    1/1     Terminating   0          3m43s
recreate-deployment-5447796b44-bdbdj    1/1     Terminating   0          3m43s
recreate-deployment-5447796b44-mkwgc    1/1     Terminating   0          3m43s
recreate-deployment-5447796b44-bdbdj    0/1     Completed     0          3m43s
recreate-deployment-5447796b44-mkwgc    0/1     Completed     0          3m43s
recreate-deployment-5447796b44-8rwv9    1/1     Terminating   0          3m43s
recreate-deployment-5447796b44-bdbdj    0/1     Completed     0          3m43s
recreate-deployment-5447796b44-bdbdj    0/1     Completed     0          3m43s
recreate-deployment-5447796b44-8rwv9    0/1     Completed     0          3m43s
recreate-deployment-74c9cd87f-fxhqh     0/1     Pending       0          0s
recreate-deployment-74c9cd87f-gjdhh     0/1     Pending       0          0s
recreate-deployment-74c9cd87f-ck65t     0/1     Pending       0          0s
recreate-deployment-74c9cd87f-fxhqh     0/1     Pending       0          0s
recreate-deployment-74c9cd87f-gjdhh     0/1     Pending       0          0s
recreate-deployment-74c9cd87f-ck65t     0/1     Pending       0          0s
recreate-deployment-74c9cd87f-fxhqh     0/1     ContainerCreating   0          0s
recreate-deployment-74c9cd87f-gjdhh     0/1     ContainerCreating   0          0s
recreate-deployment-74c9cd87f-ck65t     0/1     ContainerCreating   0          0s
recreate-deployment-74c9cd87f-gjdhh     0/1     ContainerCreating   0          1s
recreate-deployment-74c9cd87f-fxhqh     0/1     ContainerCreating   0          1s
recreate-deployment-74c9cd87f-ck65t     0/1     ContainerCreating   0          1s
recreate-deployment-5447796b44-mkwgc    0/1     Completed           0          3m44s
recreate-deployment-5447796b44-mkwgc    0/1     Completed           0          3m44s
recreate-deployment-5447796b44-8rwv9    0/1     Completed           0          3m44s
recreate-deployment-5447796b44-8rwv9    0/1     Completed           0          3m44s
recreate-deployment-74c9cd87f-fxhqh     1/1     Running             0          1s
recreate-deployment-74c9cd87f-gjdhh     1/1     Running             0          1s
recreate-deployment-74c9cd87f-ck65t     1/1     Running             0          2s
```

```
root@master:~# kubectl apply -f recreate-deployment.yaml
deployment.apps/recreate-deployment configured
root@master:~# kubectl get pods
NAME                                 READY   STATUS    RESTARTS   AGE
first-deployment-6bcdbb66f6-vpngg    1/1     Running   0          13m
first-deployment-6bcdbb66f6-xrzt8    1/1     Running   0          13m
first-deployment-6bcdbb66f6-zjlwk    1/1     Running   0          13m
recreate-deployment-74c9cd87f-ck65t  1/1     Running   0          2m26s
recreate-deployment-74c9cd87f-fxhqh  1/1     Running   0          2m26s
recreate-deployment-74c9cd87f-gjdhh  1/1     Running   0          2m26s
root@master:~#
```

3. Update an existing Deployment and perform a rollback to the previous version.

Objective: Update the container image of an existing Deployment and roll back to the previous version.

Implementation Steps:

1. Use the Deployment created in Question 1 (first-deployment).
2. Verify the existing Deployment:

   kubectl set image deployment/first-deployment nginx-container=httpd:2.4
3. Check the current container image:

   kubectl rollout history deployment/first-deployment
4. Update the image from nginx:1.26 to httpd:2.4:

kubectl rollout undo deployment/first-deployment

5. Check the updated Deployment:

kubectl rollout undo deployment/first-deployment --to-revision=1

6. Verify the updated container image:

vi resource-deployment.yaml

7. Check the rollout status:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: resource-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: resource-nginx
  template:
    metadata:
      labels:
        app: resource-nginx
    spec:
      containers:
      - name: nginx-container
        image: nginx
        ports:
        - containerPort: 80
        resources:
          requests:
            memory: "100Mi"
            cpu: "500m"
          limits:
            memory: "200Mi"
```

cpu: "1"

8. Check the Deployment revision history:

   kubectl apply -f resource-deployment.yaml

9. Roll back the Deployment to the previous version:

   kubectl describe deployment resource-deployment

10. Verify the rollback:

    kubectl describe pod <pod-name>

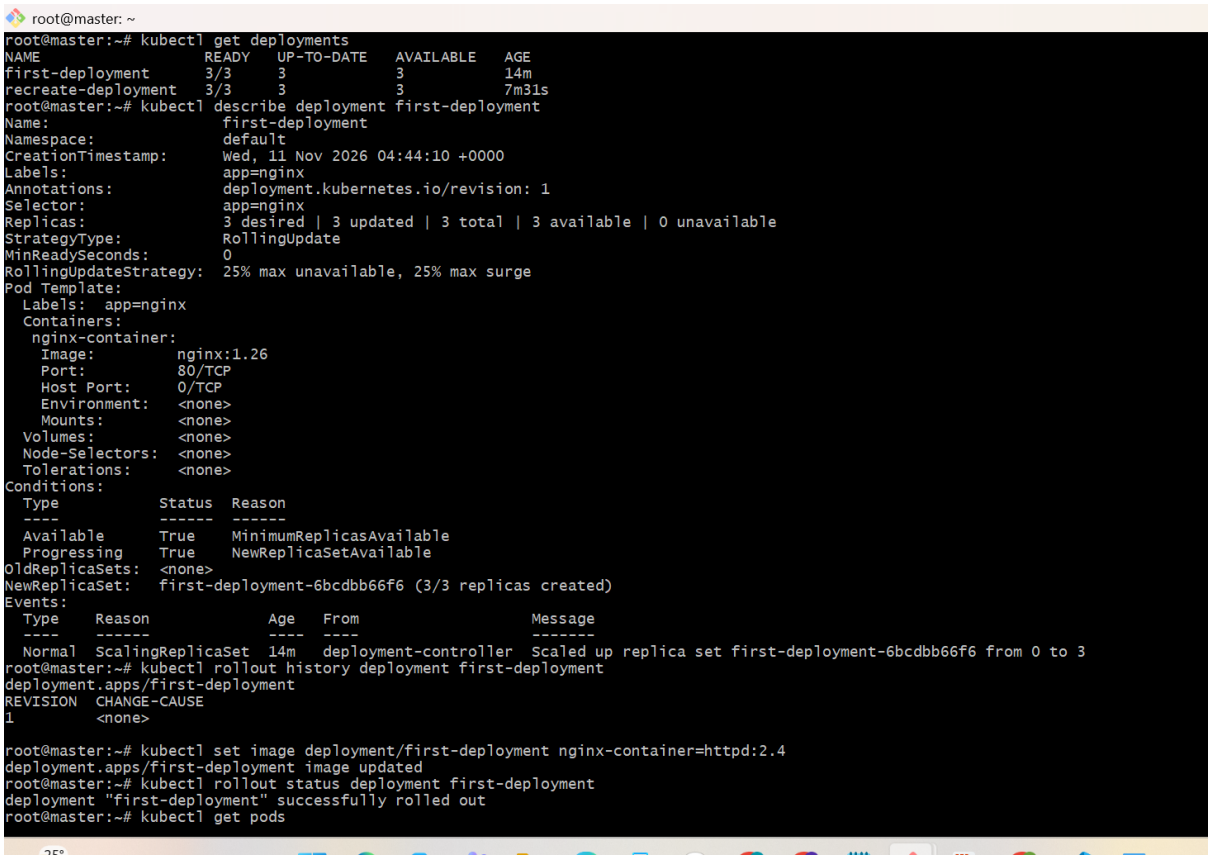
11. Check the container image again:

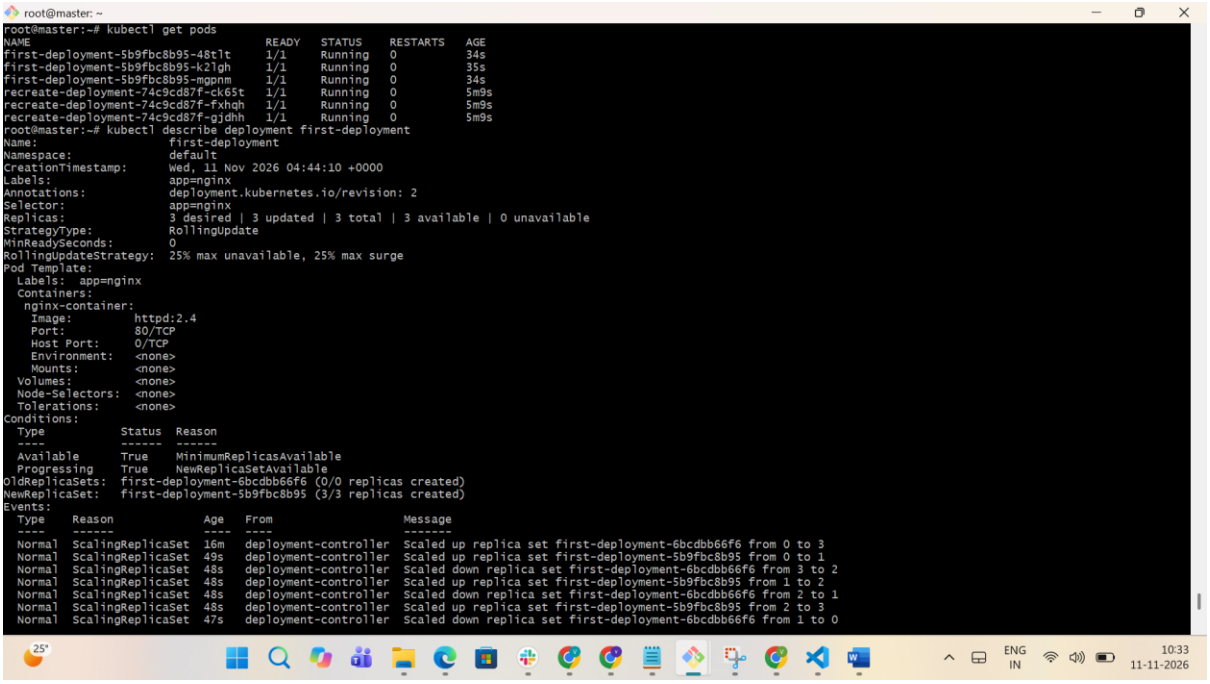
    kubectl top pods

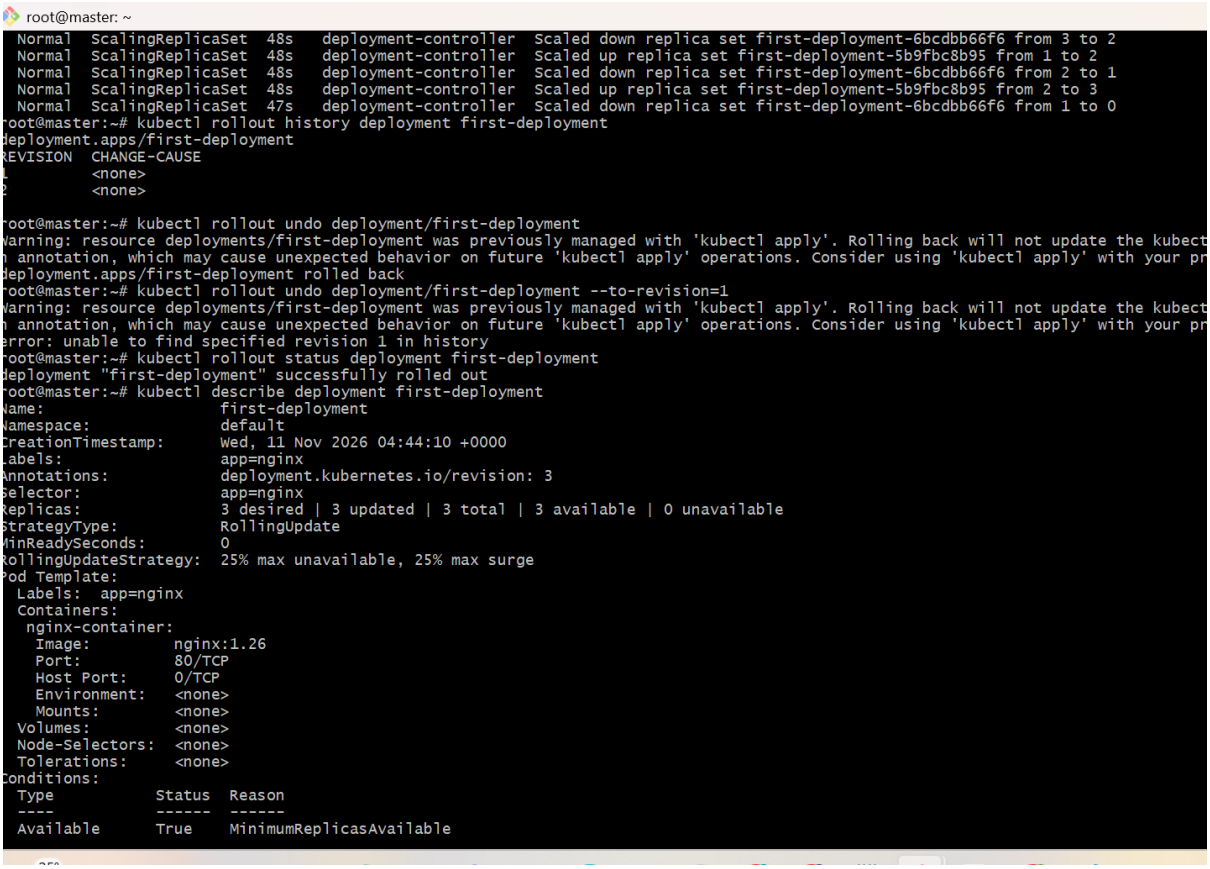
12. Verify the Pods:

    vi maxsurge-deployment.yaml

Expected Result: The Deployment image is updated to httpd:2.4. After executing the rollback command, Kubernetes restores the previous Deployment configuration.







4. Modify a Deployment to add resource requests and limits for CPU and memory.

Objective: Create a Deployment with CPU and memory requests and limits for its containers.

Implementation Steps:

1. Create a YAML file:

   kubectl apply -f maxsurge-deployment.yaml

2. Add the following configuration:

   kubectl set image deployment/maxsurge-deployment nginx-container=nginx:1.27

3. Save the file and create the Deployment:

   kubectl rollout status deployment/maxsurge-deployment

4. Verify the Deployment:

   kubectl describe deployment maxsurge-deployment

5. Verify the Pods:

   vi revision-deployment.yaml

6. Check the resource configuration:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: revision-deployment
spec:
  replicas: 3
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app: revision-nginx
  template:
    metadata:
      labels:
        app: revision-nginx
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.26
        ports:
```

- containerPort: 80

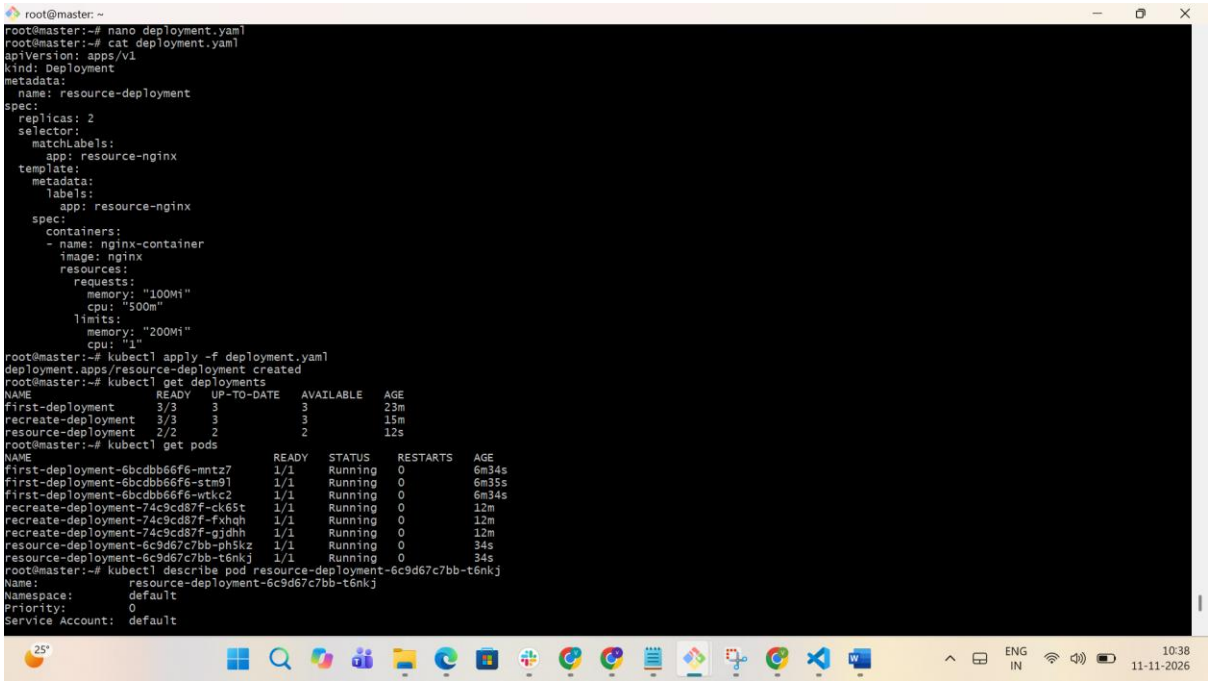
7. Check the resource configuration of a Pod:

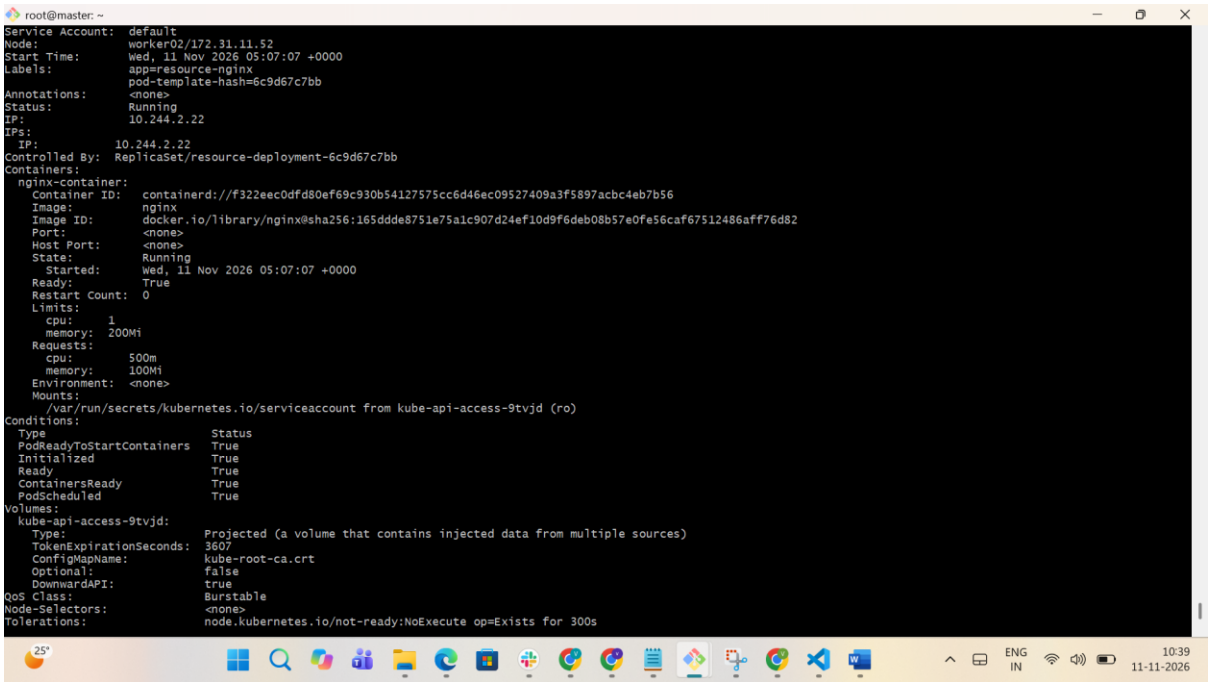
   kubectl apply -f revision-deployment.yaml

8. If Metrics Server is installed, check actual resource usage:

   kubectl set image deployment/revision-deployment nginx-container=httpd:2.4

   Expected Result: The Deployment creates two Nginx Pods with the specified CPU and memory requests and limits.





5. Create a Deployment with MaxSurge and MaxUnavailable configurations.

Objective: Configure maxSurge and maxUnavailable to control how Pods are replaced during a rolling update.

Implementation Steps:

1. Create a YAML file:

   kubectl rollout status deployment/revision-deployment

2. Add the following configuration:

   kubectl set image deployment/revision-deployment nginx-container=nginx:1.27

3. Save the file and create the Deployment:

   kubectl rollout history deployment/revision-deployment

4. Verify the Deployment:

   kubectl describe deployment revision-deployment

5. Verify that three Pods are running:

   kubectl rollout pause deployment/first-deployment

6. Open another terminal and monitor the Pods:

   kubectl set image deployment/first-deployment nginx-container=nginx:1.27

7. Update the image from nginx:1.26 to nginx:1.27:

   kubectl rollout resume deployment/first-deployment

8. Observe how Kubernetes creates new Pods and terminates old Pods during the update.

9. Check the rollout status:

   vi resource-pod.yaml

10. Verify the updated Pods:

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
  - name: nginx-container
    image: nginx
```

ports:

- containerPort: 80

resources:

requests:

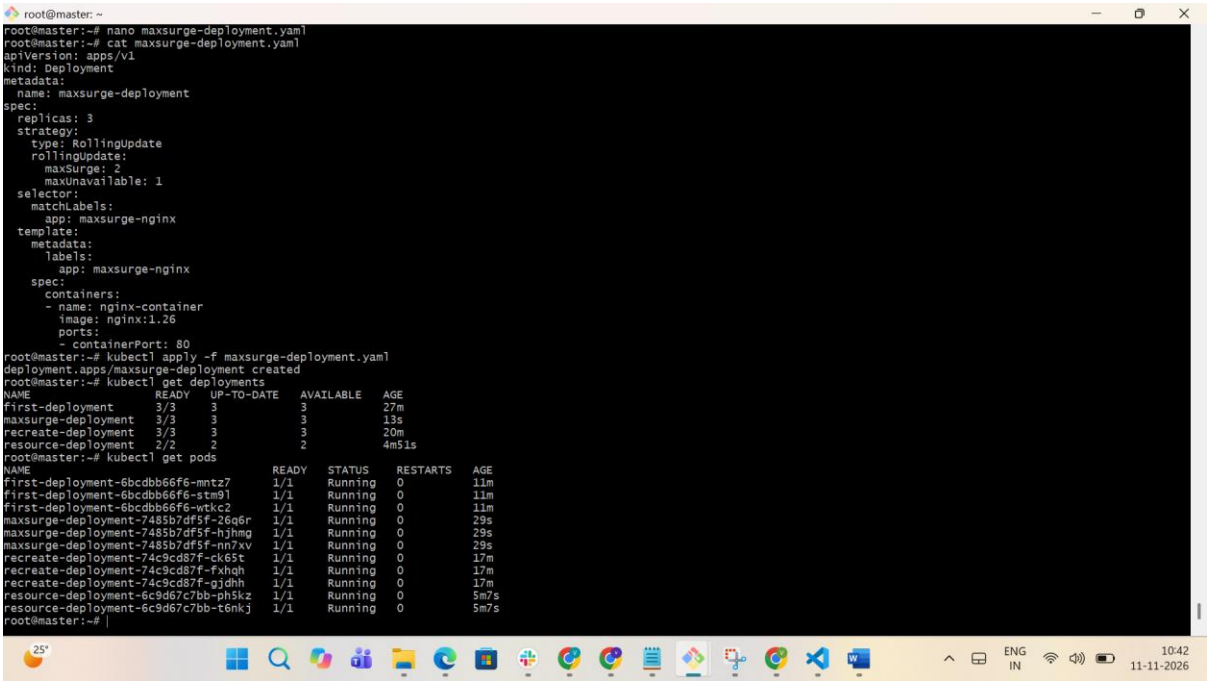
memory: "100Mi"

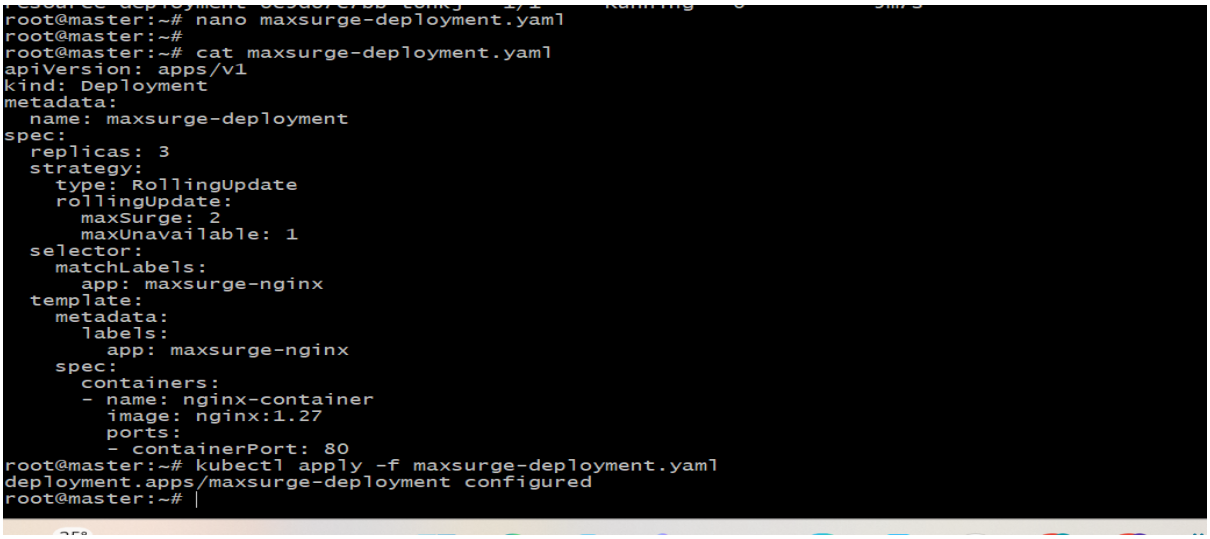
cpu: "500m"

11. Check the Deployment configuration:

kubectl apply -f resource-pod.yaml

Expected Result: Kubernetes performs a rolling update using the configured values.





```
ubuntu@master:~$ sudo su -
root@master:~# kubectl get pods -w
NAME                                   READY   STATUS    RESTARTS   AGE
first-deployment-6bcdbb66f6-mntz7      1/1     Running   0          12m
first-deployment-6bcdbb66f6-stm9l      1/1     Running   0          12m
first-deployment-6bcdbb66f6-wtkc2      1/1     Running   0          12m
maxsurge-deployment-7485b7df5f-26q6r   1/1     Running   0          2m13s
maxsurge-deployment-7485b7df5f-hjhmg   1/1     Running   0          2m13s
maxsurge-deployment-7485b7df5f-nn7xv   1/1     Running   0          2m13s
recreate-deployment-74c9cd87f-ck65t    1/1     Running   0          18m
recreate-deployment-74c9cd87f-fxhqh    1/1     Running   0          18m
recreate-deployment-74c9cd87f-gjdhh    1/1     Running   0          18m
resource-deployment-6c9d67c7bb-ph5kz   1/1     Running   0          6m51s
resource-deployment-6c9d67c7bb-t6nkj   1/1     Running   0          6m51s
maxsurge-deployment-5ffb584c6d-456rd   0/1     Pending   0          0s
maxsurge-deployment-5ffb584c6d-qjs7n   0/1     Pending   0          0s
maxsurge-deployment-5ffb584c6d-456rd   0/1     Pending   0          0s
maxsurge-deployment-7485b7df5f-26q6r   1/1     Terminating   0          3m27s
maxsurge-deployment-5ffb584c6d-qjs7n   0/1     Pending       0          0s
maxsurge-deployment-5ffb584c6d-456rd   0/1     ContainerCreating   0          0s
maxsurge-deployment-7485b7df5f-26q6r   1/1     Terminating         0          3m27s
maxsurge-deployment-5ffb584c6d-dqbnd   0/1     Pending             0          0s
maxsurge-deployment-5ffb584c6d-qjs7n   0/1     ContainerCreating   0          0s
maxsurge-deployment-5ffb584c6d-dqbnd   0/1     Pending             0          0s
maxsurge-deployment-5ffb584c6d-dqbnd   0/1     ContainerCreating   0          0s
maxsurge-deployment-7485b7df5f-26q6r   0/1     Completed           0          3m27s
maxsurge-deployment-5ffb584c6d-qjs7n   0/1     ContainerCreating   0          0s
maxsurge-deployment-5ffb584c6d-456rd   0/1     ContainerCreating   0          0s
maxsurge-deployment-5ffb584c6d-dqbnd   0/1     ContainerCreating   0          1s
maxsurge-deployment-5ffb584c6d-qjs7n   1/1     Running             0          1s
maxsurge-deployment-7485b7df5f-nn7xv   1/1     Terminating         0          3m28s
maxsurge-deployment-7485b7df5f-nn7xv   1/1     Terminating         0          3m28s
maxsurge-deployment-7485b7df5f-nn7xv   0/1     Completed           0          3m28s
maxsurge-deployment-7485b7df5f-26q6r   0/1     Completed           0          3m28s
maxsurge-deployment-7485b7df5f-26q6r   0/1     Completed           0          3m28s
maxsurge-deployment-7485b7df5f-nn7xv   0/1     Completed           0          3m28s
maxsurge-deployment-7485b7df5f-nn7xv   0/1     Completed           0          3m28s
maxsurge-deployment-5ffb584c6d-456rd   1/1     Running             0          3s
maxsurge-deployment-7485b7df5f-hjhmg   1/1     Terminating         0          3m30s
maxsurge-deployment-7485b7df5f-hjhmg   1/1     Terminating         0          3m30s
maxsurge-deployment-5ffb584c6d-dqbnd   1/1     Running             0          3s
maxsurge-deployment-7485b7df5f-hjhmg   0/1     Completed           0          3m30s
maxsurge-deployment-7485b7df5f-hjhmg   0/1     Completed           0          3m30s
maxsurge-deployment-7485b7df5f-hjhmg   0/1     Completed           0          3m30s
```

```
root@master:~# kubectl get pods
NAME                                   READY   STATUS    RESTARTS   AGE
first-deployment-6bcdbb66f6-mntz7      1/1     Running   0          15m
first-deployment-6bcdbb66f6-stm9l      1/1     Running   0          15m
first-deployment-6bcdbb66f6-wtkc2      1/1     Running   0          15m
maxsurge-deployment-5ffb584c6d-456rd   1/1     Running   0          83s
maxsurge-deployment-5ffb584c6d-dqbnd   1/1     Running   0          83s
maxsurge-deployment-5ffb584c6d-qjs7n   1/1     Running   0          83s
recreate-deployment-74c9cd87f-ck65t    1/1     Running   0          21m
recreate-deployment-74c9cd87f-fxhqh    1/1     Running   0          21m
recreate-deployment-74c9cd87f-gjdhh    1/1     Running   0          21m
resource-deployment-6c9d67c7bb-ph5kz   1/1     Running   0          9m28s
resource-deployment-6c9d67c7bb-t6nkj   1/1     Running   0          9m28s
root@master:~#
```

6. Set up a Deployment with a custom revision history limit.

Objective: Configure the number of old Deployment revisions Kubernetes should retain.

Implementation Steps:

1. Create a YAML file:

   kubectl get pods -o wide

2. Add the following configuration:

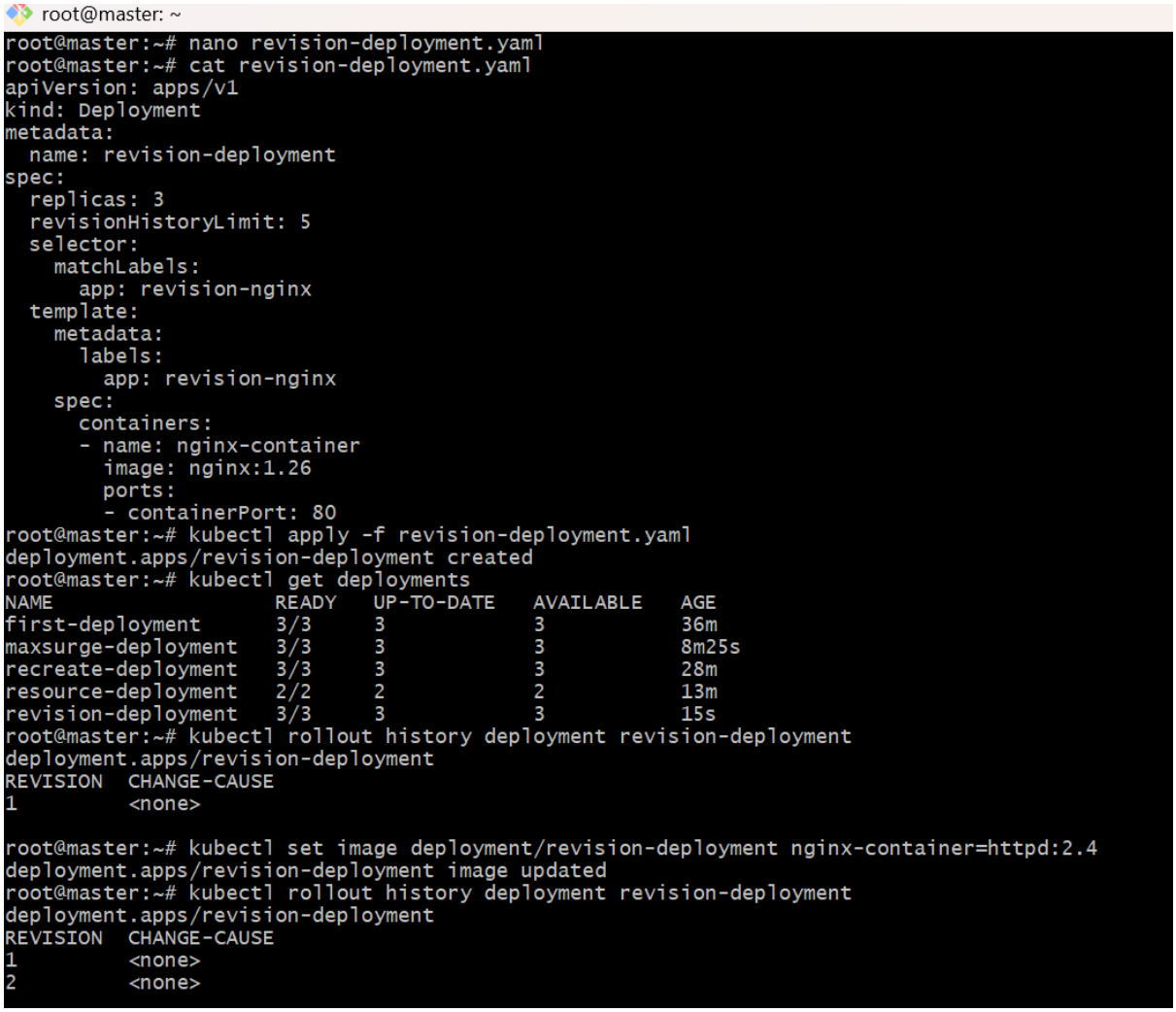
   kubectl describe pod resource-pod

3. Save the file and create the Deployment:

   kubectl top pod resource-pod --containers

4. Verify the Deployment:
5. Update the image:
6. Wait for the rollout to complete:
7. Perform another image update:
8. Check the rollout history:
9. Check the ReplicaSets:
10. Describe the Deployment:

Expected Result: Kubernetes retains up to five old ReplicaSet revisions for the Deployment, subject to available revision history. Older revisions beyond the configured retention limit can be removed.



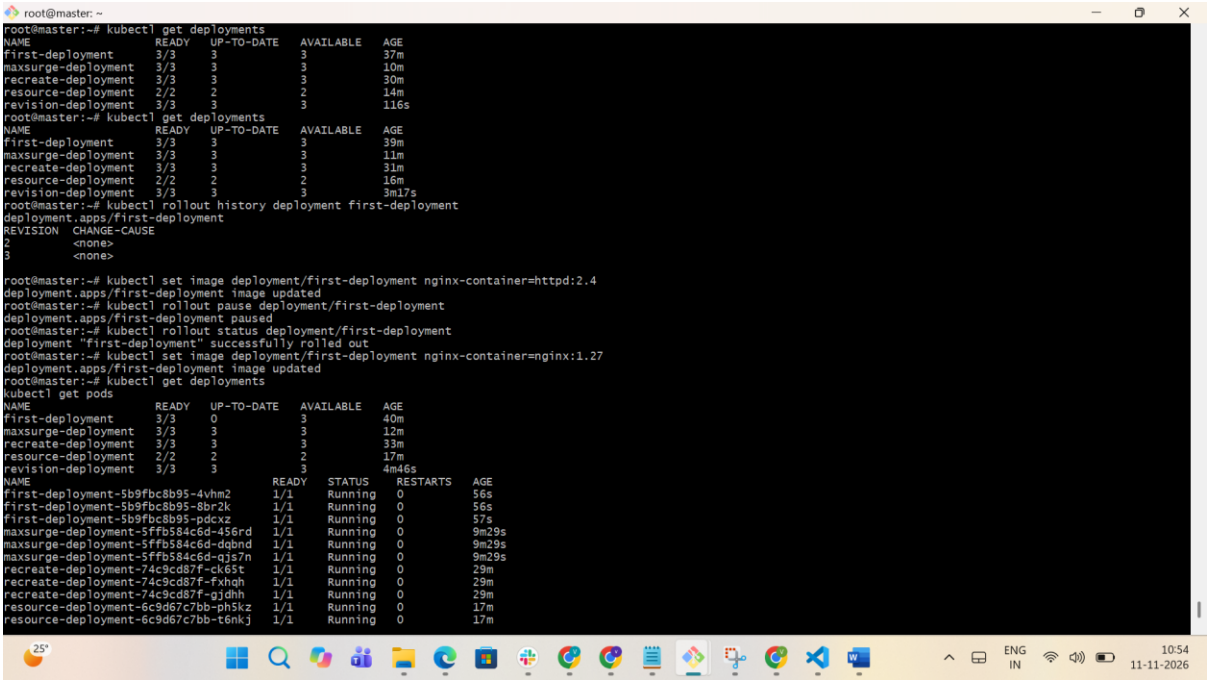
7. Pause a Deployment during an update, and then resume it.

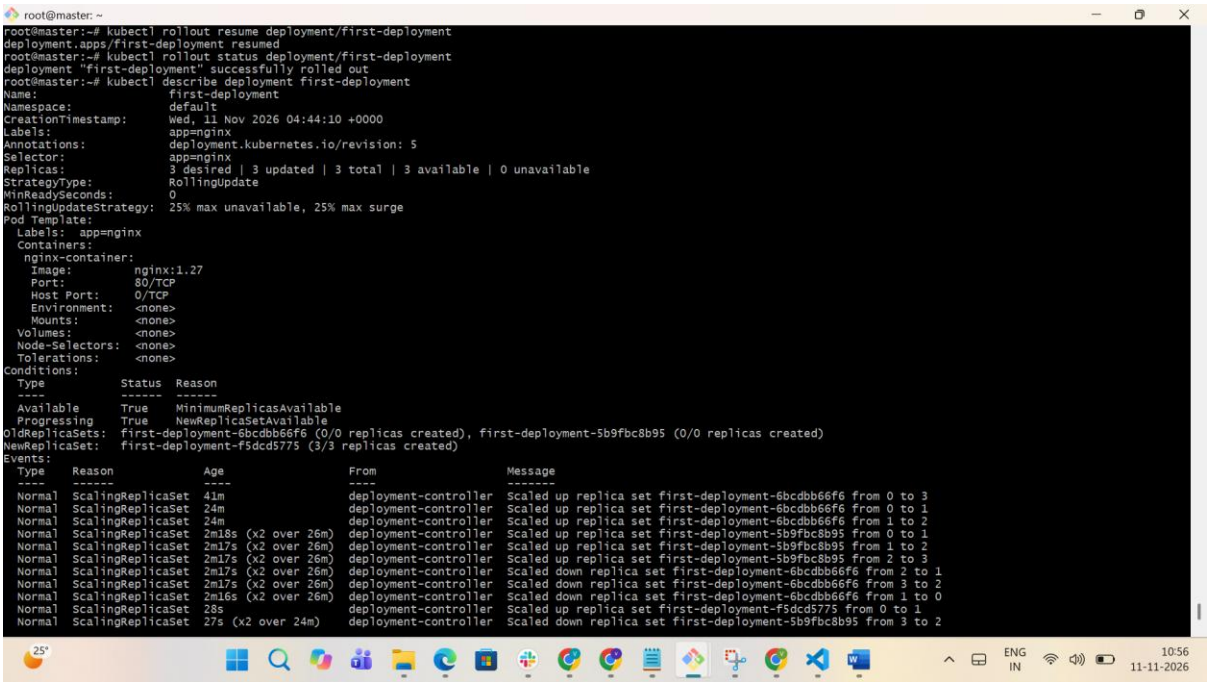
Objective: Pause a Deployment, make an image update while it is paused, and resume the Deployment.

Implementation Steps:

1. Use the Deployment created in Question 1 (first-deployment).
2. Verify the Deployment:
3. Pause the Deployment:
4. Update the container image while the Deployment is paused:
5. Check the Deployment configuration:
6. Resume the Deployment:
7. Check the rollout status:
8. Verify the updated image:
9. Verify the Pods:

Expected Result: The Deployment is paused while configuration changes are made. After resuming, Kubernetes processes the changes and completes the rollout.





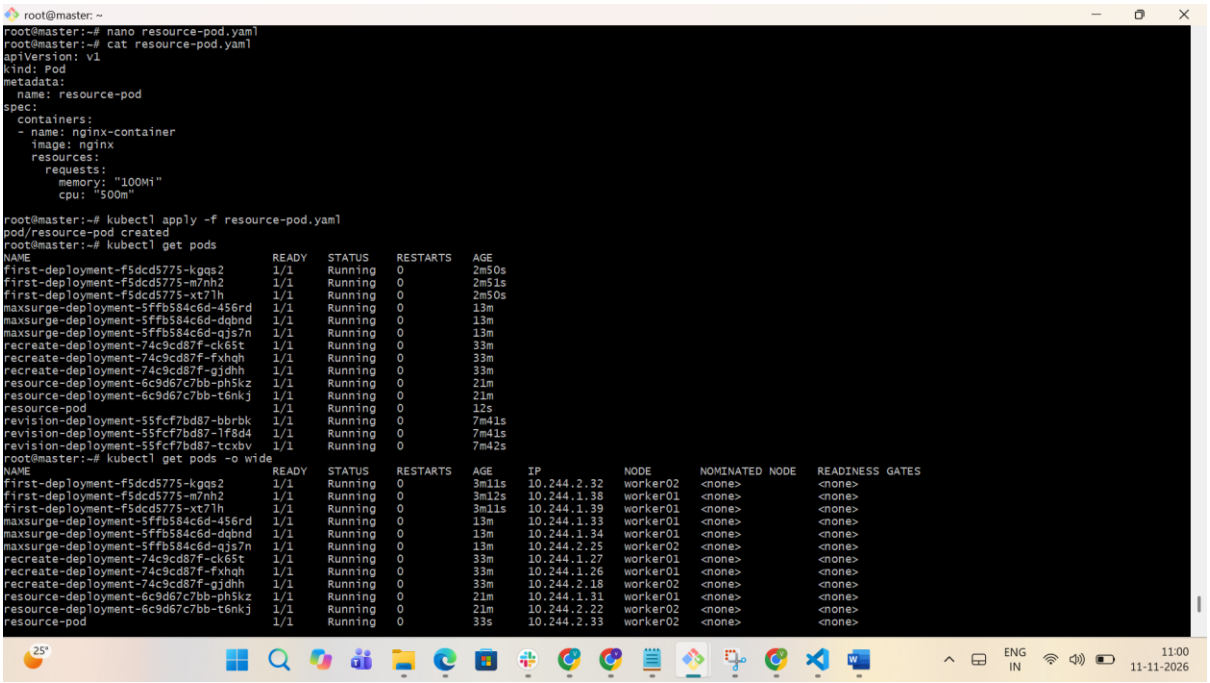
8. Create a pod using resource requests for memory and CPU, and observe how the scheduler assigns it to a node.

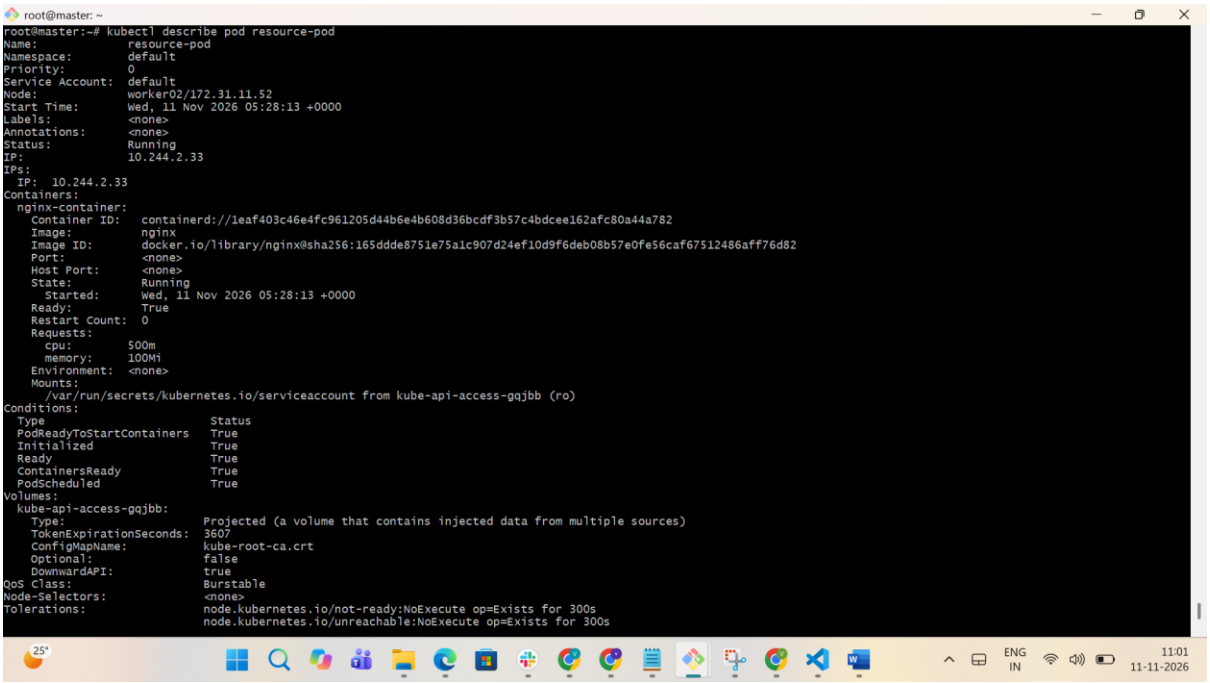
Objective: Create a Pod with CPU and memory requests and verify how the Kubernetes scheduler assigns the Pod to a node.

Implementation Steps:

1. Create a YAML file:
2. Add the following configuration:
3. Save the file and create the Pod:
4. Verify the Pod status:
5. Check the node where the Pod is scheduled:
6. Describe the Pod:
7. Check the resource requests under the container details.
8. Check the scheduling events at the bottom of the output.
9. If Metrics Server is installed, check the actual resource usage:

Expected Result: Kubernetes schedules the Pod on a node that has sufficient available CPU and memory resources to satisfy the Pod's requests.


root@master: ~
root@master:~# nano resource-pod.yaml
root@master:~# cat resource-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
  - name: nginx-container
    image: nginx
    resources:
      requests:
        memory: "100Mi"
        cpu: "500m"
root@master:~# kubectl apply -f resource-pod.yaml
pod/resource-pod created
root@master:~# kubectl get pods
NAME READY STATUS RESTARTS AGE
first-deployment-f5dcd5775-kgqs2 1/1 Running 0 2m50s
first-deployment-f5dcd5775-m7nh2 1/1 Running 0 2m51s
first-deployment-f5dcd5775-xt7lh 1/1 Running 0 2m50s
maxsurge-deployment-5ffb584c6d-456rd 1/1 Running 0 13m
maxsurge-deployment-5ffb584c6d-dqbnd 1/1 Running 0 13m
maxsurge-deployment-5ffb584c6d-qjs7n 1/1 Running 0 13m
recreate-deployment-74c9cd87f-ck65t 1/1 Running 0 33m
recreate-deployment-74c9cd87f-fxhqh 1/1 Running 0 33m
recreate-deployment-74c9cd87f-gjdhh 1/1 Running 0 33m
resource-deployment-6c9d67c7bb-ph5kz 1/1 Running 0 21m
resource-deployment-6c9d67c7bb-t6nkj 1/1 Running 0 21m
resource-pod 1/1 Running 0 12s
revision-deployment-55fcf7bd87-bbrbk 1/1 Running 0 7m41s
revision-deployment-55fcf7bd87-lf8d4 1/1 Running 0 7m41s
revision-deployment-55fcf7bd87-tcxbv 1/1 Running 0 7m42s
root@master:~# kubectl get pods -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
first-deployment-f5dcd5775-kgqs2 1/1 Running 0 3m11s 10.244.2.32 worker02 <none> <none>
first-deployment-f5dcd5775-m7nh2 1/1 Running 0 3m12s 10.244.1.38 worker01 <none> <none>
first-deployment-f5dcd5775-xt7lh 1/1 Running 0 3m11s 10.244.1.39 worker01 <none> <none>
maxsurge-deployment-5ffb584c6d-456rd 1/1 Running 0 13m 10.244.1.33 worker01 <none> <none>
maxsurge-deployment-5ffb584c6d-dqbnd 1/1 Running 0 13m 10.244.1.34 worker01 <none> <none>
maxsurge-deployment-5ffb584c6d-qjs7n 1/1 Running 0 13m 10.244.2.25 worker02 <none> <none>
recreate-deployment-74c9cd87f-ck65t 1/1 Running 0 33m 10.244.1.27 worker01 <none> <none>
recreate-deployment-74c9cd87f-fxhqh 1/1 Running 0 33m 10.244.1.26 worker01 <none> <none>
recreate-deployment-74c9cd87f-gjdhh 1/1 Running 0 33m 10.244.2.18 worker02 <none> <none>
resource-deployment-6c9d67c7bb-ph5kz 1/1 Running 0 21m 10.244.1.31 worker01 <none> <none>
resource-deployment-6c9d67c7bb-t6nkj 1/1 Running 0 21m 10.244.2.22 worker02 <none> <none>
resource-pod 1/1 Running 0 33s 10.244.2.33 worker02 <none> <none>
25°
ENG
IN
11:00
11-11-2026


root@master: ~
root@master:~# kubectl describe pod resource-pod
Name: resource-pod
Namespace: default
Priority: 0
Service Account: default
Node: worker02/172.31.11.52
Start Time: Wed, 11 Nov 2026 05:28:13 +0000
Labels: <none>
Annotations: <none>
Status: Running
IP: 10.244.2.33
IPs:
IP: 10.244.2.33
Containers:
nginx-container:
Container ID: containerd://1eaf403c46e4fc961205d44b6e4b608d36bcdf3b57c4bdcee162afc80a44a782
Image: nginx
Image ID: docker.io/library/nginx@sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
Port: <none>
Host Port: <none>
State: Running
Started: Wed, 11 Nov 2026 05:28:13 +0000
Ready: True
Restart Count: 0
Requests:
cpu: 500m
memory: 100Mi
Environment: <none>
Mounts:
/var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-gqjbb (ro)
Conditions:
Type Status
PodReadyToStartContainers True
Initialized True
Ready True
ContainersReady True
PodScheduled True
Volumes:
kube-api-access-gqjbb:
Type: Projected (a volume that contains injected data from multiple sources)
TokenExpirationSeconds: 3607
ConfigMapName: kube-root-ca.crt
Optional: false
DownwardAPI: true
QoS Class: Burstable
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
25°
ENG
IN
11:01
11-11-2026